TileMoE: Stage-Adaptive Load-Aware Scheduling for Communication-Computation Fused MoE

Yuan Zhang, Wenquan Xu, Lei Liu, Tianran Leng, Chenyi Wen, Chunyi Zhou,
Ruoyu Zhou, Wenbo Sun, Xuan Zhong
Huawei Technologies Co., Ltd.

***Abstract*—Mixture-of-Experts (MoE) architectures expand model capacity through sparse activation, yet distributing experts across devices tightly couples communication and computation. Fused megakernels overlap these operations. However, our analysis of an Ascend A5 baseline reveals residual inefficiencies in cube utilization, intermediate data movement, tile load balance, and accumulation timing. These inefficiencies point to a scheduling mismatch. Fixed execution policies cannot adequately accommodate changing resource demands and readiness conditions. We propose TileMoE, a stage-adaptive, load-aware scheduling design that reconfigures execution roles and assigns tile work according to data readiness and core progress. Its three-stage pipeline overlaps data transfer with computation, returns expert outputs in batches, and starts accumulation once a token's results are available. We evaluate TileMoE against a fused AscendC MegaMoE baseline under A8W8, with EP=8, topk=8, and 16 experts per card. Measured execution time decreases by 3.69%–7.15% in decode and 13.76%–21.93% in prefill.**

***Index Terms*—Mixture-of-Experts, communication-computation fusion, stage-adaptive scheduling, load-aware scheduling, NPU**

I. Introduction

Mixture-of-Experts (MoE) architectures scale model capacity by selectively activating experts for each token [1]–[3]. Under expert parallelism, however, this sparsity introduces communication dependencies between devices. Tokens must reach their selected experts, and the resulting outputs must return to their source devices for reduction. The execution chain therefore couples communication with expert computation. Dispatch supplies inputs to the first grouped matrix multiplication (GMM1), which computes the up and gate projections before SwiGLU activation. The second grouped matrix multiplication (GMM2) computes the down projection, after which combine returns expert outputs for weighted TopK accumulation.

Executing this chain as separate kernels incurs repeated HBM transfers for intermediate results. Fused megakernel designs [4] mitigate this overhead by integrating the stages and overlapping their execution. On GPUs, DeepSeek's MegaMoE [5], [6] combines expert waves, fine-grained warp specialization, and on-chip storage. Ascend NPUs expose a different execution structure, with AI cube cores (AIC) and AI vector cores (AIV) operating through independent instruction streams. AscendC MegaMoE [7] exploits this structure. It assigns matrix computation to AIC and dispatch, SwiGLU, and combine to AIV, while retaining the GMM1-to-SwiGLU handoff on chip. These advances establish fusion as a foundation for further scheduling improvements.

However, fusion alone does not ensure sustained utilization throughout the pipeline. Our profiling and analysis of AscendC MegaMoE on an Ascend A5 NPU identify four opportunities for improvement. Cube idle gaps motivate tighter coordination between dispatch and downstream computation. The GMM2-to-combine handoff still passes through HBM, calling for an on-chip path with a compatible return layout. Meanwhile, static tile quotas can leave early-finishing cores without additional work. Finally, TopK accumulation starts late, even when some tokens' selected expert results have already returned.

These inefficiencies point to a scheduling mismatch: fixed execution policies do not adequately accommodate changing resource demands and readiness conditions across the fused MoE pipeline. This observation motivates TileMoE, a stage-adaptive, load-aware scheduling design. Across stages, TileMoE reconfigures execution roles as the mix of communication, matrix computation, and accumulation changes. Within each stage, it assigns ready tiles according to core progress. The design also coordinates result placement and accumulation. Metadata records where results will return, allowing combine to transfer them in batches while the source device tracks each token's readiness for accumulation.

Our contributions are summarized as follows:

- We analyze residual inefficiencies in an already fused MoE operator, identifying a mismatch between fixed execution policies and changing resource demands and readiness conditions (Section III).
- We propose stage-adaptive, load-aware scheduling that coordinates execution roles with readiness- and progress-driven tile assignment. The design supports batched result return, an on-chip GMM2 handoff, and early TopK accumulation (Section IV).
- We evaluate TileMoE against a fused AscendC MegaMoE baseline under one A8W8 configuration. Results show measured execution-time reductions across decode and prefill workloads, together with the remaining limitations (Section V).

II. Background

A. The MoE operator chain

Fig. 1 illustrates the execution chain of a distributed MoE layer under expert parallelism. Each token selects *topk* experts,



Fig. 1. End-to-end MoE operator chain, from dispatch through expert computation to combine and weighted TopK accumulation.

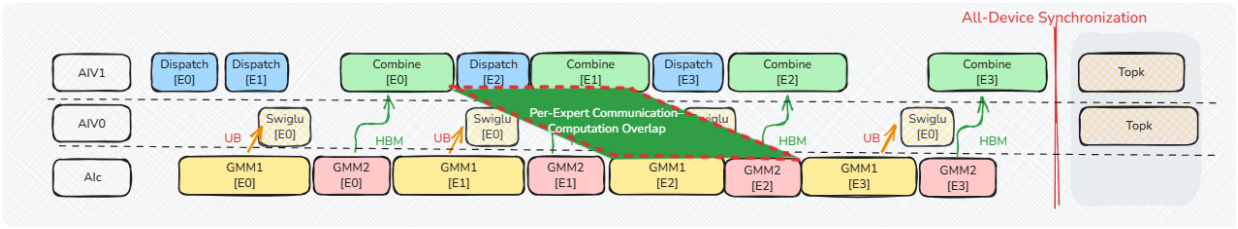


Fig. 2. The fused AscendC MegaMoE pipeline. Work advances expert by expert, and stages belonging to different experts overlap in time.

and dispatch transfers its activations to the corresponding devices. GMM1, SwiGLU, and GMM2 compute the expert outputs. Combine then returns these outputs to the source device, where TopK accumulation reduces them using the routing weights.

Local efficiency alone is insufficient. End-to-end performance depends on sustained overlap between communication and computation, motivating the integration of these phases into a single kernel.

B. The fused baseline

AscendC MegaMoE [7] integrates dispatch, expert computation, and combine into a fused pipeline, replacing multi-kernel serial execution with fine-grained overlap. The organization in Fig. 2 allows work from different experts to advance concurrently.

The pipeline exploits the complementary capabilities of AIC and AIV. AIC execute GMM1 and GMM2, while AIV handle dispatch transfers, SwiGLU activation, and combine write-back. Each expert follows the dispatch–GMM1–SwiGLU–GMM2–combine dependency chain. Across experts, the phases overlap so that communication proceeds concurrently with matrix computation.

The two AIC-to-AIV handoffs use different data paths. GMM1 transfers its intermediate results directly to SwiGLU through the on-chip cube-vector path and unified buffer (CV/UB). GMM2, by comparison, writes its outputs to HBM before AIV reads them for combine. This second handoff therefore incurs an additional off-chip write and read.

We adopt this fused implementation as the baseline for our analysis and evaluation.

C. Programming substrate

TileMoE is implemented in PTO-ISA (Parallel Tile Operation Instruction Set Architecture), a tile-level virtual instruction set for Ascend NPUs [8]. PTO-ISA exposes explicit control over computation, data movement, and on-chip placement. A common load/store interface expresses transfers to local HBM or remote device memory, while a tile-to-tile move represents cube-to-vector handoffs. This abstraction lets the scheduler coordinate tile execution with data placement and transfer dependencies.

III. Inefficiencies That Remain After Fusion

To examine the remaining inefficiencies after fusion, we profile the baseline pipeline on an Ascend A5 NPU. Fig. 3 separates cube and vector execution along a shared time axis. The trace exhibits several cube idle gaps, extended vector idle intervals, and a TopK accumulation phase concentrated near the end of execution.

Together with the baseline structure, these observations motivate an examination of two data handoffs, intra-matmul tile assignment, and the condition that initiates accumulation.

Handoff 1, dispatch to GMM1: input readiness. GMM1 execution depends on the availability of dispatched inputs. An otherwise available AIC must wait when its next tile's inputs have not arrived, consistent with gaps on the cube timeline. Prioritizing dispatch over later combine work provides a way to sustain this input supply.

Handoff 2, GMM2 to combine: an HBM round trip. Staging GMM2 outputs through HBM introduces an intermediate write and read. An on-chip handoff must also match combine's consumption rate and return layout. GMM2 produces expert-major tiles, whereas final accumulation accesses selected expert results for each token. Restoring token order during return requires separate placement of each routed result, whereas preserving expert-major segments requires a mapping for later token-wise access. Efficient direct transfer therefore calls for coordinated layout and scheduling.

Within GMM1 and GMM2: fixed tile quotas. Static assignment predetermines each core's share of work. When cores progress at different rates, early finishers cannot take over work from unfinished quotas. Progress-driven assignment allows those cores to continue with ready tiles.

At the end of the pipeline: TopK accumulation starts too late. The baseline trace places accumulation near the end of execution. Yet each token becomes reducible once its selected expert results have returned, even while combine continues for other tokens. Tracking this finer readiness condition allows accumulation to overlap outstanding GMM2 and combine work.

Together, these constraints motivate stage-adaptive, load-aware scheduling. Execution roles must follow the changing



Fig. 3. Hardware pipeline trace of the fused AscendC baseline. Cube idle gaps, extended vector-core idle intervals, and a late TopK accumulation start are visible on the timeline.

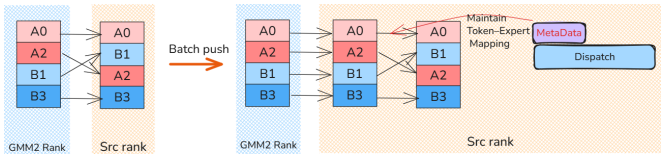


Fig. 4. Route metadata connects expert-major return positions to token-major accumulation, allowing combine to return contiguous multi-row segments.

mix of work, while actual readiness and progress guide tile assignment and accumulation.

IV. Stage-Adaptive Load-Aware Scheduling

We consider an A8W8 configuration in which dispatch and the grouped matmuls consume FP8 inputs, while combine returns BF16 results. Excluding scales, metadata, and alignment, their activation and result payload volumes are approximately in a 1:2 ratio. GMM1 jointly computes the up and gate projections from a hidden size of 7168, producing 2048 outputs per branch and 4096 in total. GMM2 projects 2048 inputs back to 7168 outputs. The main matrix arithmetic of GMM1 and GMM2 is therefore in a 2:1 ratio.

TileMoE coordinates scheduling at two complementary levels. Across stages, it reconfigures AIC and AIV roles as input preparation, expert computation, and accumulation become executable. Within a stage, it assigns ready tiles according to core progress. The mechanisms below establish when work can begin and where data is placed, forming the three-stage pipeline in Section IV-E.

A. Separating dispatch in time from combine

To prioritize input preparation, TileMoE overlaps dispatch with GMM1 at the start of execution. GMM2 and combine wait until their entry dependencies are satisfied. A designated subset of AIV0 subblocks performs dispatch, while others prepare metadata. By checking readiness for each tile, GMM1 consumes available inputs before all dispatch work finishes, preserving overlap during this initial stage.

GMM2 becomes eligible once local dispatch has completed and the route metadata required for direct return is ready. This separates dispatch from combine on each device while retaining dispatch–GMM1 overlap. Input preparation thus takes priority before result return begins.

B. A metadata phase, batched combine, and the on-chip path

The key layout choice is to separate result return from token-order restoration. GMM2 produces expert-major tiles, and combine retains this ordering in the source device's return buffer. Final accumulation uses a position mapping to locate each token's selected results. Return transfers can therefore preserve contiguous expert-major segments, leaving token-order restoration to accumulation.

The fused kernel receives expert IDs and routing weights after gate and TopK selection. AIV0 workers use these inputs to construct an expert-major route ordering and its inverse in source-local GM workspace. For token t and expert slot j, $\texttt{expandedRowIdx}[t \cdot \text{topk} + j]$ identifies the corresponding row in the return buffer. The map stores one 32-bit entry per route, occupying $4M \cdot \text{topk}$ bytes before alignment for M input tokens. UB provides temporary processing and caching. Additional metadata includes source-rank segment offsets and tile-task descriptors for return and scheduling.

Metadata preparation overlaps dispatch, and its readiness is checked before GMM2 enters the direct-return path. Combine then uses the prepared offsets to intersect each GMM2 tile with source-rank row segments. Each contiguous intersection is transferred as multiple rows. The inverse map in Fig. 4 lets final accumulation gather expert-major rows in token order.

Batched return allows GMM2 to pass its results to combine on chip through CV/UB. On the direct peer-window path, combine consumes these results without first writing them



Fig. 5. Concurrent GMM1 and GMM2 execution with an on-chip GMM2-to-combine handoff on the direct return path.

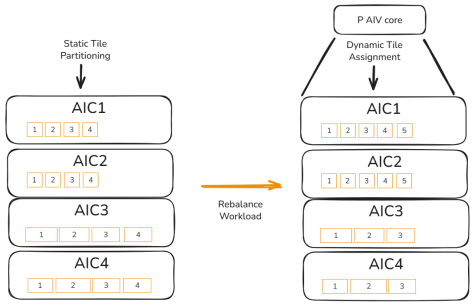


Fig. 6. The producer assigns ready tile tasks according to consumer progress, subject to each core's scheduling phase.

to local HBM and reading them back. TopK accumulation later reads the source-device return buffer. In the concurrent schedule of Fig. 5, AICs outside the GMM2 group continue GMM1 computation. Section IV-E describes their allocation and progress-driven assignment.

C. Producer-consumer tile dispatch

TileMoE separates task assignment from execution through an AIV0 producer interacting with AIC consumers. Through the interaction depicted in Fig. 6, the producer tracks input availability and core progress, then assigns ready tasks to eligible AICs. GMM1 can start when dispatched inputs arrive, whereas GMM2 waits for the preceding computation to finish. When an AIC completes a tile, it reports progress to the producer and receives subsequent ready work. Early finishers can thus continue within the current scheduling phase. This interaction makes data readiness and execution progress the basis of load-aware assignment.

A single AIV0 subblock serves as producer. Initial GMM1 tiles are assigned statically so that AICs can begin computation while scheduling information is being prepared. Once the initial input-layout metadata is ready, the producer coordinates subsequent work according to AIC progress. This combines static assignment at startup with progress-driven task distribution.

D. Early TopK accumulation for ready tokens

A token becomes ready for accumulation once all of its selected expert results have returned, without requiring combine to finish globally. TileMoE tracks expert completion progress and checks each token's requirements before gathering rows for weighted reduction. Fig. 7 depicts the resulting overlap, with available vector workers accumulating ready tokens while GMM2 and combine continue.

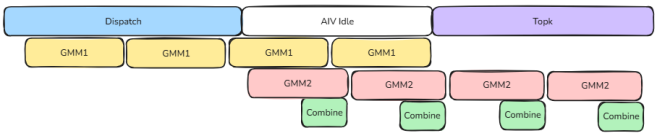


Fig. 7. The illustrated schedule starts TopK accumulation after the GMM1 handoff, allowing accumulation for ready tokens to overlap remaining GMM2 and combine work.

For larger workloads, accumulation begins with a subset of AIV0 workers after the GMM1 group handoff. The coordinator provides an initial snapshot of expert completion progress so these workers can identify ready tokens. Other workers join after completing their earlier tasks, expanding the resources available for accumulation. Each token is still checked for readiness before reduction. This arrangement limits early overlap with GMM1 while allowing accumulation to proceed alongside the remaining GMM2 and combine work.

E. The three-stage pipeline

The preceding mechanisms form a three-stage pipeline that adapts the active mix of work as dependencies are satisfied. The 36-AIC configuration in Fig. 8 uses a 24:12 GMM1-to-GMM2 split in the middle stage. Host-side tiling determines core-group sizes. Runtime scheduling distributes ready tiles and coordinates role handoffs, allowing individual cores to transition as their dependencies permit.

Stage 1: dispatch and GMM1. All AICs initially execute GMM1 work. AIV0 subblocks prepare dispatched inputs and metadata, with one dedicated to the producer, while paired AIV1 subblocks process SwiGLU. GMM1 computes each tile as its inputs become ready.

Stage 2: GMM1, GMM2, and combine. The configured GMM2 group transitions after completing its initial GMM1 work and satisfying the entry dependencies. These include local dispatch completion, expanded route metadata, source-rank return offsets, and consumer readiness on the direct return path. Paired AIV1 subblocks finish SwiGLU and consume GMM2 outputs for combine through CV/UB. Other AICs continue GMM1 computation, establishing overlap between the two matmuls and result return.

Stage 3: GMM2, combine, and TopK. Once the remaining GMM1 work has been assigned, its consumers finish their current tasks. The AIV0 producer then directs available AICs to GMM2 as its entry conditions are satisfied. Released AIV workers accumulate ready tokens while GMM2 and combine continue. Other workers join after completing their roles, progressively shifting vector resources toward final accumulation.

Configured allocation and runtime adaptation. In the illustrated 36-AIC configuration, AIV0[0–23] performs dispatch, AIV0[24–34] prepares metadata, and AIV0[35] serves as producer. The default middle-stage AIC split is 24:12. Shape-specific tiling entries use 22:14 for EP=8, 16 experts per card, and input row counts $M = 1024$ or 2048. These allocations are selected before kernel execution. At runtime, stage transitions change each group's work, while load-aware scheduling distributes ready tiles according to consumer

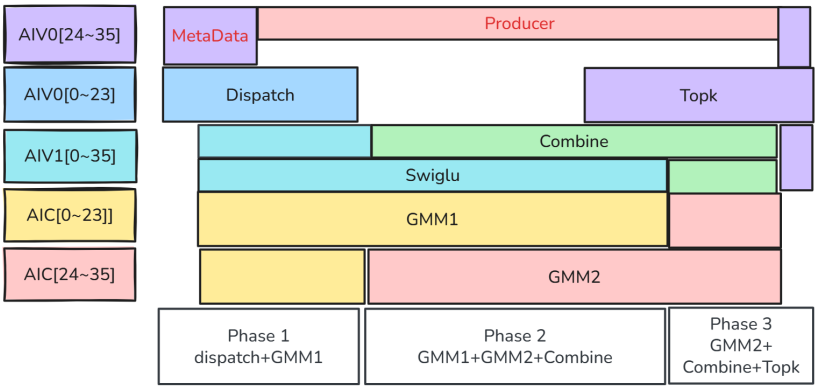


Fig. 8. Three-stage scheduling with an illustrative 24:12 core-group split. Execution roles change as dependencies and per-core handoffs permit.

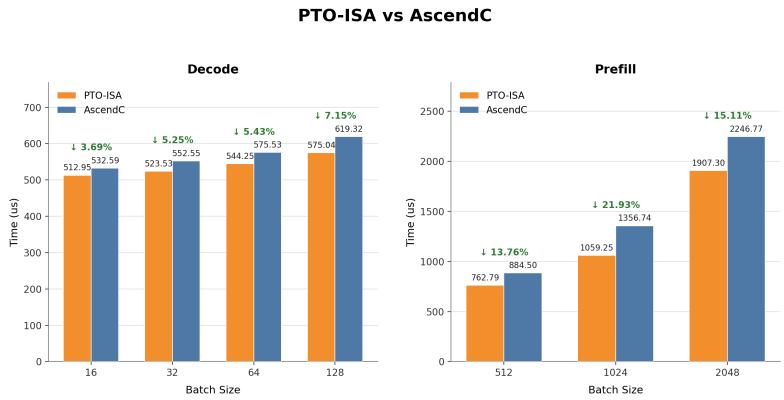


Fig. 9. Measured execution time and percentage reductions relative to the fused AscendC MegaMoE baseline. PTO-ISA denotes TileMoE.

TABLE I
PIPELINE STEPS AND THE KEY PTO-ISA PRIMITIVES USED

Pipeline step	Primitive	Boundary
Dispatch and combine	`TLOAD`/`TSTORE`	device to device
Weight/activation staging	`TLOAD`/`TSTORE`	HBM to on-chip
GMM2 to combine handoff	`TMOV`	cube to vector
Grouped matmul tiles	`TMATMUL_MX`	compute
Quantization	`TQUANT`	compute
TopK conversion, accumulation	`TCVT`/`TAXPY`	compute

progress. The AIV0 producer coordinates these transitions with AIC consumers through readiness checks and task assignments. Paired AIV1 subblocks follow the corresponding computation.

F. Primitives used in the implementation

Table I summarizes the PTO-ISA primitives used in the pipeline. A common `TLOAD`/`TSTORE` interface expresses local HBM staging and inter-device transfers, including dispatch and direct combine return. The `TMOV` primitive represents the on-chip GMM2-to-combine handoff. These primitives connect tile execution to the pipeline's data movement and buffer placement.

V. EVALUATION

A. Setup

We evaluate TileMoE against the early-August 2026 code baseline of CANN AscendC MegaMoE [7]. Both implementations execute identical workloads on the same Ascend A5 hardware, using the layer shapes in Section IV. The A8W8 configuration uses EP=8, topk=8, and 16 experts per card. Following the Batch Size labels in Fig. 9, decode uses 16, 32, 64, and 128, while prefill uses 512, 1024, and 2048. We report measured execution time in microseconds. The percentage reduction is $(T_{\text{baseline}} - T_{\text{TileMoE}})/T_{\text{baseline}} \times 100\%$. The PTO-ISA label in the figure denotes the TileMoE implementation.

B. Results

The measurements in Fig. 9 show lower execution time for TileMoE than for the fused baseline across all displayed Batch Size values. The reductions range from 3.69% to 7.15% for decode and from 13.76% to 21.93% for prefill.

Decode. The relative reduction is smaller in decode but grows across the evaluated Batch Size values. Larger batches provide more tiles for the producer to distribute among AICs, creating additional opportunities for load balancing. This trend is consistent with the progress-driven scheduling design.

Prefill. The relative reductions are larger in prefill, where more token computation and result return must be coordinated across the pipeline. The largest reduction is 21.93% at Batch Size 1024. Unlike decode, the relative benefit does not grow monotonically with Batch Size and declines at 2048.

C. Post-optimization trace

Fig. 10 provides a qualitative view of the resulting overlap. Cube execution becomes more continuous, while TopK accu-



Fig. 10. Hardware execution trace of TileMoE's three-stage pipeline.

mulation proceeds concurrently with remaining GMM2 and combine work. This behavior is consistent with the intended three-stage schedule.

Residual cube and vector idle intervals remain near the execution tail. Nevertheless, overlapping accumulation with unfinished GMM2 and combine reduces the accumulation work exposed after result return completes.

VI. Conclusion

This work presents TileMoE, a stage-adaptive, load-aware scheduling design for communication-computation fused MoE. It addresses the mismatch between fixed execution policies and changing resource demands and readiness conditions. Across stages, TileMoE coordinates execution-role transitions, while within each stage, it assigns ready tiles according to core progress. Result-layout metadata supports batched return, while token readiness tracking allows accumulation to overlap unfinished computation and communication. Under the evaluated A8W8 configuration, this coordinated pipeline reduces measured execution time by 3.69%–7.15% in decode and 13.76%–21.93% in prefill relative to the fused AscendC MegaMoE baseline.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1701.06538

[2] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://arxiv.org/abs/2006.16668

[3] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 1280–1297. [Online]. Available: https://aclanthology.org/2024.acl-long.70/

[4] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu, "COMET: Fine-grained computation-communication overlapping for mixture-of-experts," *Proceedings of Machine Learning and Systems*, vol. 7, 2025. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2025/hash/e27ea0cd50b798ff8942caf9203f0992-Abstract-Conference.html

[5] DeepSeek AI, "DeepGEMM: Fused moe and megamoe kernel implementations," Open-source software repository, 2026, accessed 2026-10-13. [Online]. Available: https://github.com/deepseek-ai/DeepGEMM

[6] DeepSeek-AI *et al.*, "DeepSeek-V4: Towards highly efficient million-token context intelligence," 2026, version 1; Section 3.1: Fine-Grained Communication-Computation Overlap in Expert Parallelism. [Online]. Available: https://arxiv.org/html/2606.19348v1

[7] CANN, "MegaMoE in ops-transformer/mc2/mega_moe," Official CANN operator documentation and source code, 2026, accessed 2026-10-13; historical performance baseline revision is not identified by this reference. [Online]. Available: https://gitcode.com/cann/ops-transformer/tree/master/mc2/mega_moe

[8] ——, "PTO Tile Library: Parallel tile operation virtual instruction set architecture," Official source code and documentation, 2026, accessed 2026-10-13. [Online]. Available: https://gitcode.com/cann/pto-isa